

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería
Inteligencia Artificial
Semestre I – 2026



Excelencia que trasciende

DELVALLE
GRUPO EDUCATIVO

Proyecto 2

Integrantes:

Calderón Argueta, Wilson Alejandro – 22018
Nájera Marakovits, Ingrid Nina Alessandra – 231088
Antón Escobar, José Alejandro – 221041

Índice

1. Introducción	2
2. Problema 1: Generación aleatoria de laberintos	2
2.1. Descripción de los algoritmos	2
2.1.1. Algoritmo de Kruskal	2
2.1.2. Algoritmo de Prim	2
2.2. Detalles de implementación	3
2.3. Visualizaciones	3
3. Problema 2: Solución de un laberinto 60×80	4
3.1. Descripción de los algoritmos de búsqueda	4
3.1.1. BFS	4
3.1.2. DFS	4
3.1.3. Dijkstra	4
3.1.4. A*	5
3.2. Detalles de implementación	5
3.3. Visualizaciones	5
4. Problema 3: Comparación en 25 laberintos 45×55	9
4.1. Metodología	9
4.2. Visualizaciones por escenario	9
4.3. Resultados globales	10
5. Conclusiones	11

1. Introducción

Este proyecto tiene como objetivo comparar el desempeño de distintos algoritmos de búsqueda aplicados a la resolución de laberintos generados de forma aleatoria. Para ello se abordaron tres problemas: la generación de laberintos mediante algoritmos de árbol de expansión mínima, la solución de un laberinto de tamaño fijo usando cuatro algoritmos de búsqueda, y la comparación sistemática de esos algoritmos en múltiples escenarios distintos.

El código fue desarrollado en Python, usando estructuras de datos clásicas como colas, pilas y montículos, además de la librería `matplotlib` para generar las visualizaciones. El repositorio del proyecto se encuentra en: <https://github.com/Ninaswiftie09/IA/tree/Proyecto2>

2. Problema 1: Generación aleatoria de laberintos

2.1. Descripción de los algoritmos

Se implementaron dos algoritmos para generar laberintos aleatorios perfectos, es decir, laberintos donde existe exactamente un camino entre cualquier par de celdas.

2.1.1. Algoritmo de Kruskal

El algoritmo de Kruskal genera un árbol de expansión mínima sobre el grafo de celdas del laberinto. El proceso es el siguiente: primero se crea una lista de todas las paredes que separan celdas adyacentes y se mezcla de forma aleatoria. Luego, para cada pared en la lista, se verifica si las dos celdas que separa pertenecen a componentes distintos usando una estructura Union-Find. Si es así, se elimina la pared y se unen los dos componentes. El proceso continúa hasta que todas las celdas forman un solo componente conexo.

La estructura Union-Find utilizada implementa compresión de caminos y unión por rango, lo que le da una complejidad casi lineal. Esto hace que Kruskal sea muy eficiente para laberintos grandes.

2.1.2. Algoritmo de Prim

El algoritmo de Prim construye el laberinto de forma incremental, partiendo de una celda inicial aleatoria. En cada paso, se elige al azar una pared de la lista de paredes candidatas, es decir, paredes que separan una celda ya visitada de una celda no visitada. Si la celda del otro lado no ha sido visitada, se elimina la pared y se agrega esa celda al laberinto, sumando sus paredes adyacentes a la lista de candidatas.

A diferencia de Kruskal, Prim construye el laberinto de forma más local, expandiéndose desde una celda central, lo que produce estructuras visualmente distintas.

2.2. Detalles de implementación

El laberinto se representa como una matriz de tamaño $(2M + 1) \times (2N + 1)$, donde M y N son las filas y columnas de celdas respectivamente. Las celdas se ubican en posiciones de índice impar y las paredes en posiciones de índice par. Un valor de 1 representa una pared y un valor de 0 representa un pasillo.

Ambos algoritmos cuentan con soporte para captura de fotogramas intermedios, lo que permite visualizar el proceso de construcción paso a paso.

2.3. Visualizaciones

La figura 1 muestra cuatro estados intermedios del proceso de construcción para cada algoritmo: el estado inicial, el 25 %, el 50 % y el estado final. Se puede observar cómo Kruskal elimina paredes de forma dispersa en todo el espacio, mientras que Prim tiende a crecer de forma más concentrada desde el punto de partida.

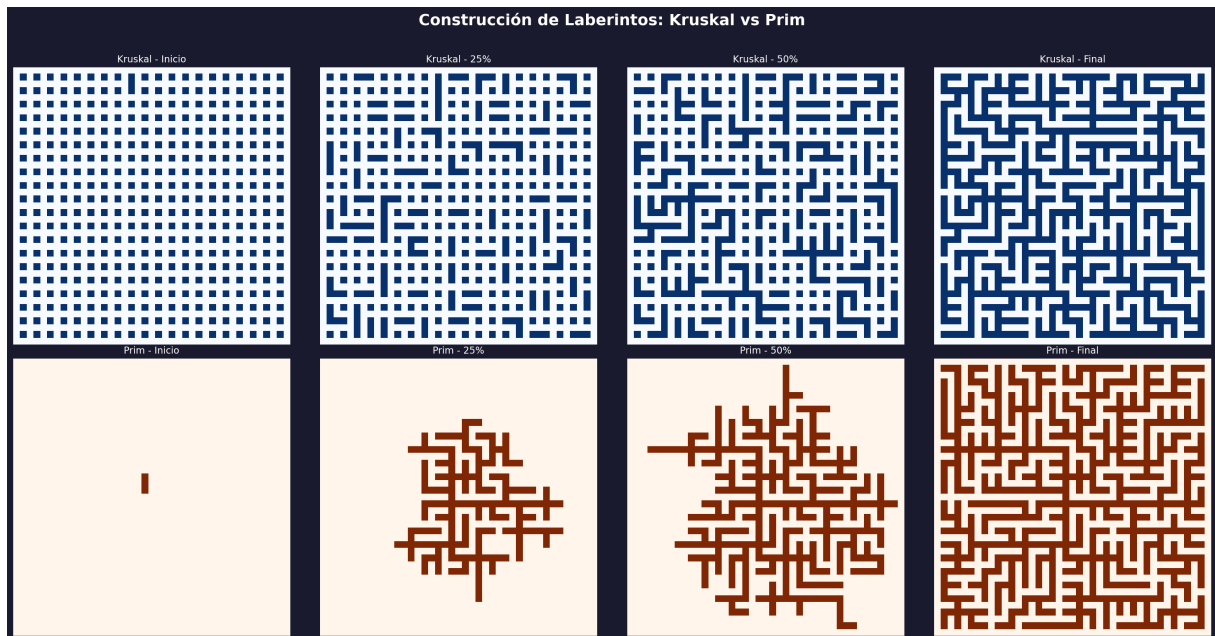


Figura 1: Proceso de construcción de laberintos con Kruskal y Prim en distintas etapas.

La figura 2 muestra los laberintos completos generados por cada algoritmo sobre una cuadrícula de 25×30 celdas. Las diferencias visuales son notables: Kruskal produce laberintos con pasillos más distribuidos de manera uniforme, mientras que Prim genera estructuras con ramificaciones más orgánicas.

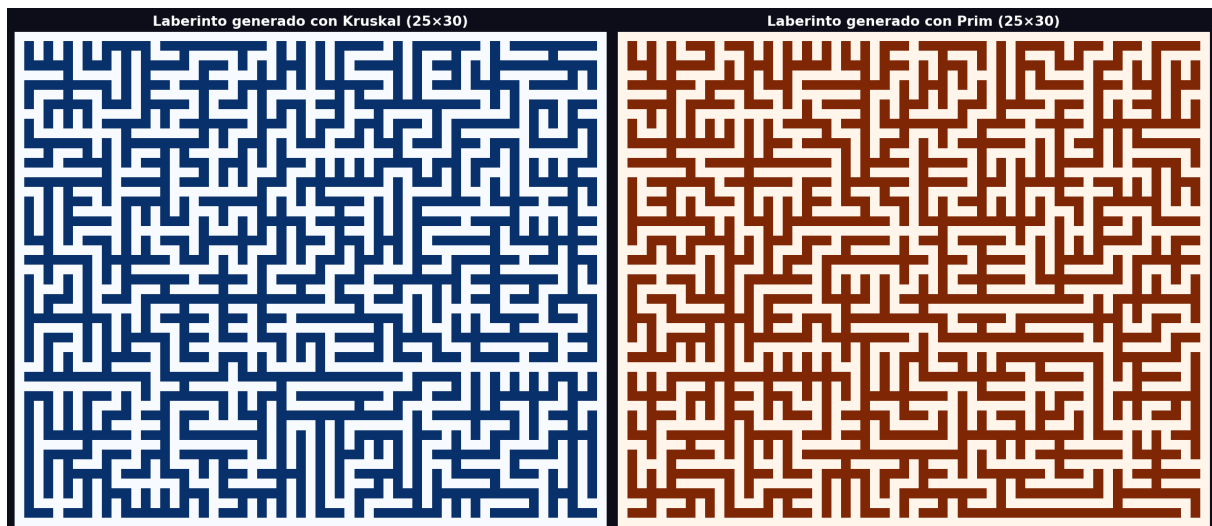


Figura 2: Laberintos finales generados con Kruskal y con Prim sobre una cuadrícula de 25×30 celdas.

3. Problema 2: Solución de un laberinto 60×80

3.1. Descripción de los algoritmos de búsqueda

Se implementaron cuatro algoritmos para encontrar la ruta más corta entre la celda de entrada y la celda de salida del laberinto.

3.1.1. BFS

La búsqueda en anchura explora el grafo nivel por nivel, comenzando desde el nodo inicial. Utiliza una cola para mantener el orden de exploración y garantiza encontrar el camino más corto en grafos no ponderados. Su desventaja es que puede explorar una gran cantidad de nodos antes de alcanzar la meta.

3.1.2. DFS

La búsqueda en profundidad explora tan lejos como sea posible por cada rama antes de retroceder. Utiliza una pila para gestionar el orden de exploración. No garantiza el camino más corto, pero puede ser eficiente en laberintos donde la solución se encuentra en la dirección en que el algoritmo avanza primero.

3.1.3. Dijkstra

El algoritmo de Dijkstra explora los nodos en orden de costo acumulado desde el inicio, usando una cola de prioridad. En grafos donde todas las aristas tienen el mismo peso, como los laberintos de este proyecto, produce el mismo resultado que BFS pero con una gestión de prioridades diferente.

3.1.4. A*

El algoritmo A* combina el costo acumulado desde el inicio con una heurística que estima el costo restante hasta la meta. En este proyecto se usó la distancia Manhattan como heurística, dado que el movimiento en el laberinto es solo en las cuatro direcciones cardinales. Esto le permite guiar la búsqueda hacia la meta de forma más directa, reduciendo el número de nodos explorados sin perder la garantía de encontrar el camino óptimo.

3.2. Detalles de implementación

Todos los algoritmos comparten la función `get_neighbors`, que retorna las celdas adyacentes transitables desde una posición dada, y la función `reconstruct_path`, que reconstruye el camino desde la meta hasta el inicio siguiendo los punteros de padres almacenados durante la búsqueda.

La entrada del laberinto es la celda $(1, 1)$ en coordenadas del grid expandido, que corresponde a la esquina superior izquierda. La salida es la celda $(H - 2, W - 2)$, que corresponde a la esquina inferior derecha, donde H y W son las dimensiones del grid expandido.

3.3. Visualizaciones

Las figuras 3, 4, 5 y 6 muestran el resultado de cada algoritmo sobre el mismo laberinto de 60×80 celdas. Las celdas exploradas aparecen en azul claro, el camino encontrado en amarillo, el punto de inicio en verde y la meta en rojo.

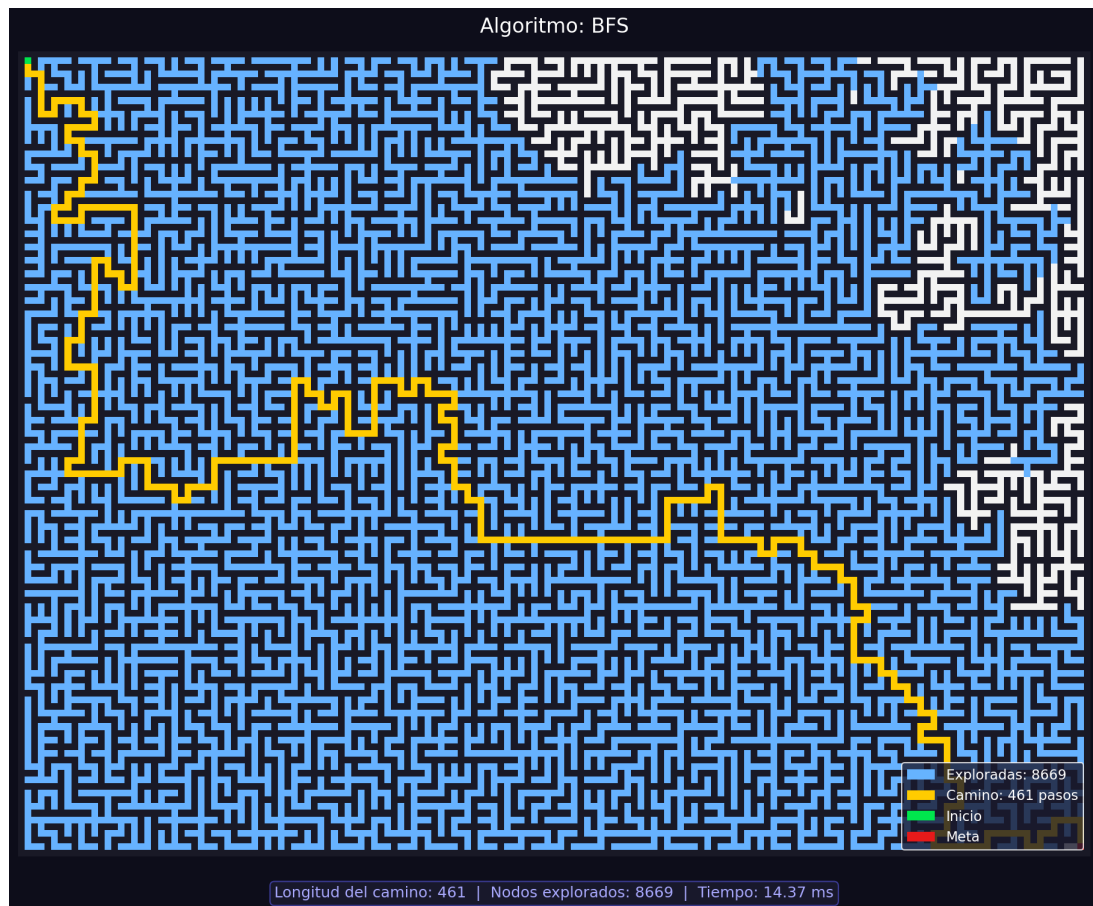


Figura 3: Solución del laberinto 60×80 con BFS.



Figura 4: Solución del laberinto 60×80 con DFS.

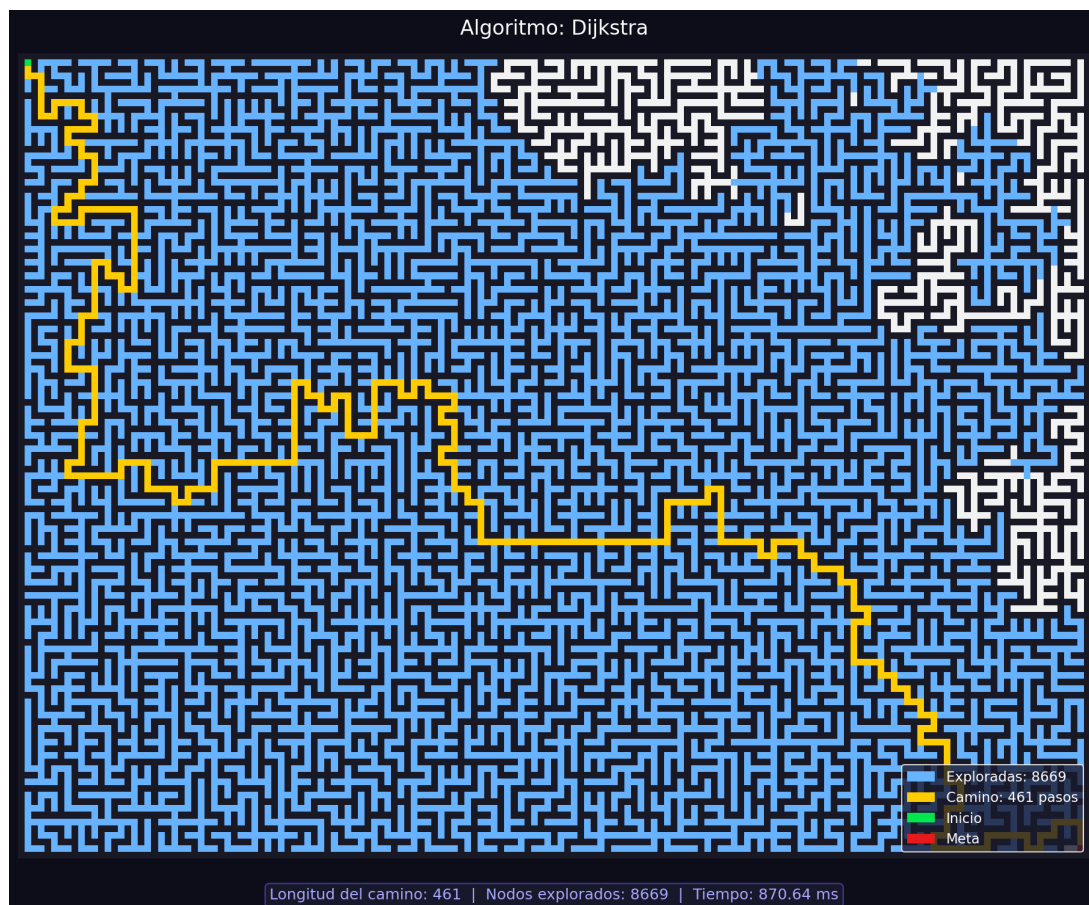


Figura 5: Solución del laberinto 60×80 con Dijkstra.

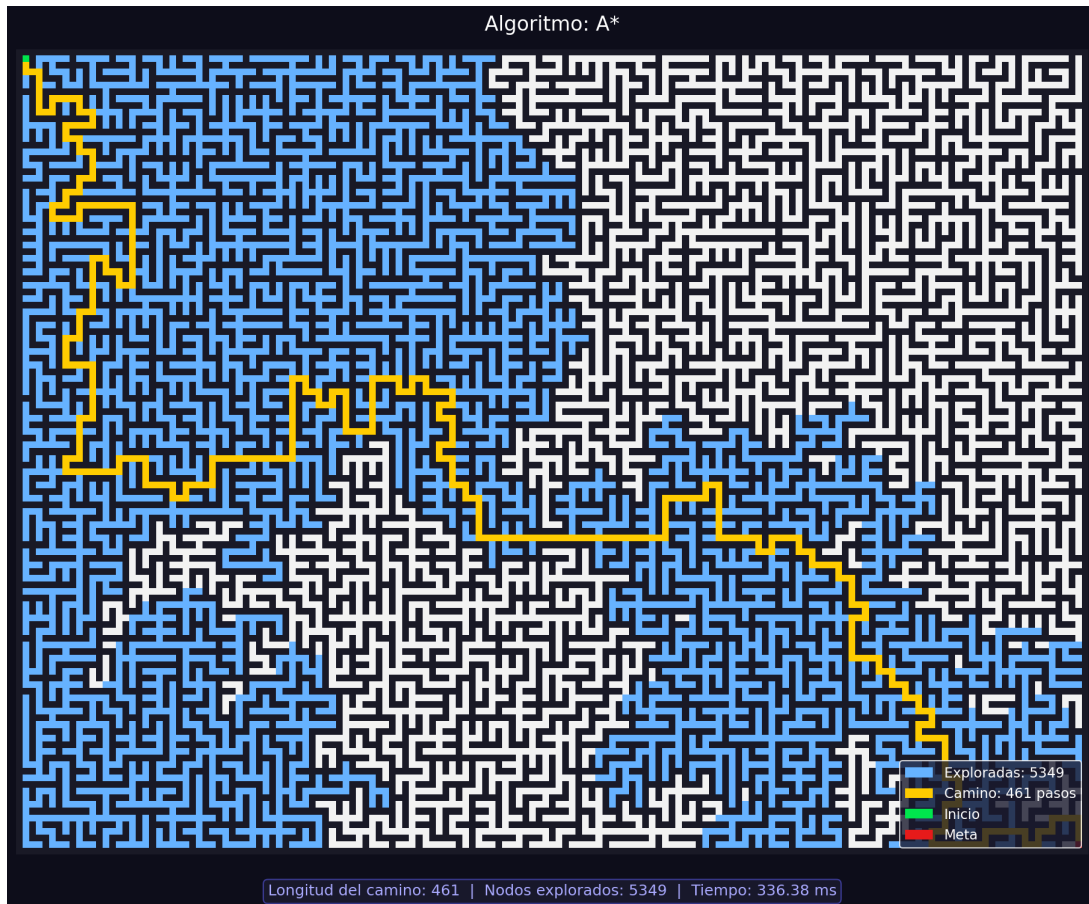


Figura 6: Solución del laberinto 60×80 con A*.

4. Problema 3: Comparación en 25 laberintos 45×55

4.1. Metodología

Se generaron 25 laberintos aleatorios de 45×55 celdas, alternando entre los algoritmos de Kruskal y Prim. En cada laberinto se seleccionaron aleatoriamente una posición de inicio A y una posición de meta B , garantizando que la distancia Manhattan entre ambas fuera de al menos 10 unidades.

Para cada laberinto se ejecutaron los cuatro algoritmos de búsqueda y se registraron tres métricas: la cantidad de nodos explorados, el tiempo de ejecución y la longitud del camino encontrado. Los algoritmos fueron ordenados del 1 al 4 en cada escenario según el número de nodos explorados, siendo 1 el que menos exploró. Al final se calculó el ranking promedio de cada algoritmo en los 25 escenarios.

4.2. Visualizaciones por escenario

La figura 7 muestra seis de los veinticinco escenarios evaluados. En cada uno se presenta el laberinto con la región explorada por A* en azul y el camino encontrado en amarillo, junto con estadísticas de comparación frente a los demás algoritmos.

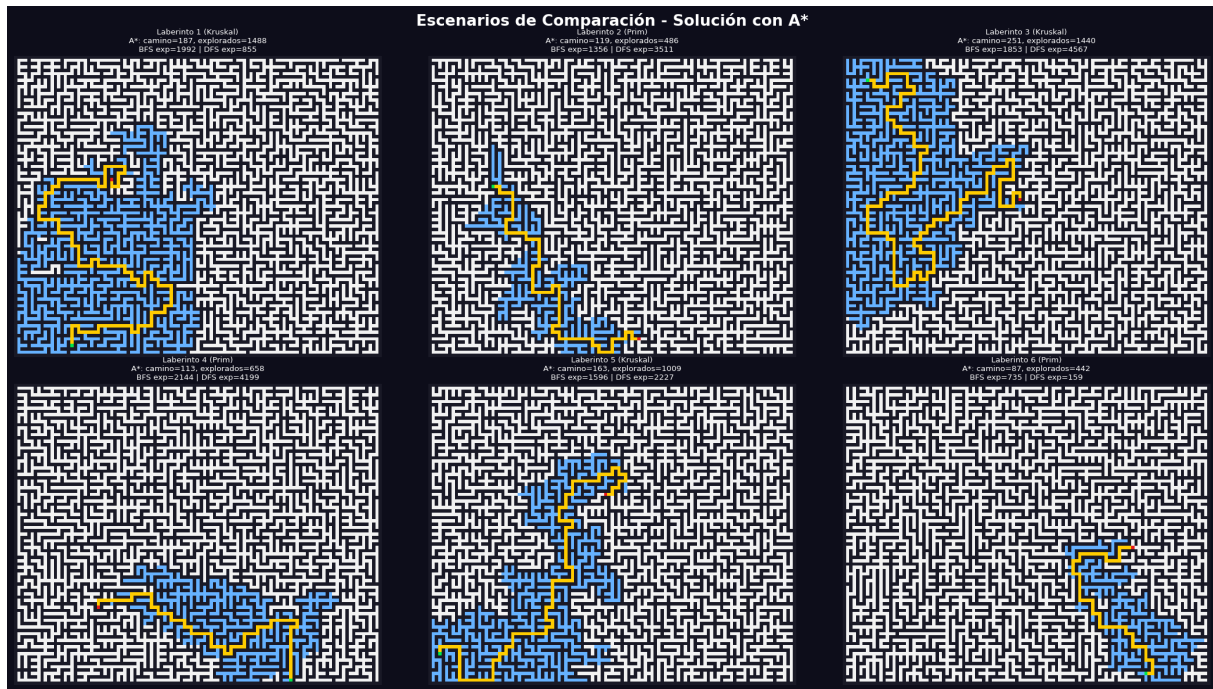


Figura 7: Muestra de seis escenarios del Problema 3, resueltos con A*.

4.3. Resultados globales

La figura 8 presenta la tabla resumen con el ranking promedio, el promedio de nodos explorados, el tiempo de ejecución y la longitud del camino para los cuatro algoritmos, ordenados de mejor a peor desempeño general.

Tabla Resumen: Comparación de Algoritmos (25 Laberintos 45×55)				
Algoritmo	Rank Promedio	Explorados (avg)	Tiempo avg (ms)	Longitud camino (avg)
A*	1.16	1331.2	36.036	172.5
DFS	2.72	2340.8	48.354	172.5
BFS	2.92	2336.0	3.636	172.5
Dijkstra	3.20	2339.9	80.492	172.5

Figura 8: Tabla resumen del ranking promedio de los cuatro algoritmos en los 25 laberintos.

La figura 9 muestra gráficas de barras que comparan los promedios de nodos explorados, tiempo de ejecución y longitud del camino para cada algoritmo a lo largo de todos los escenarios.

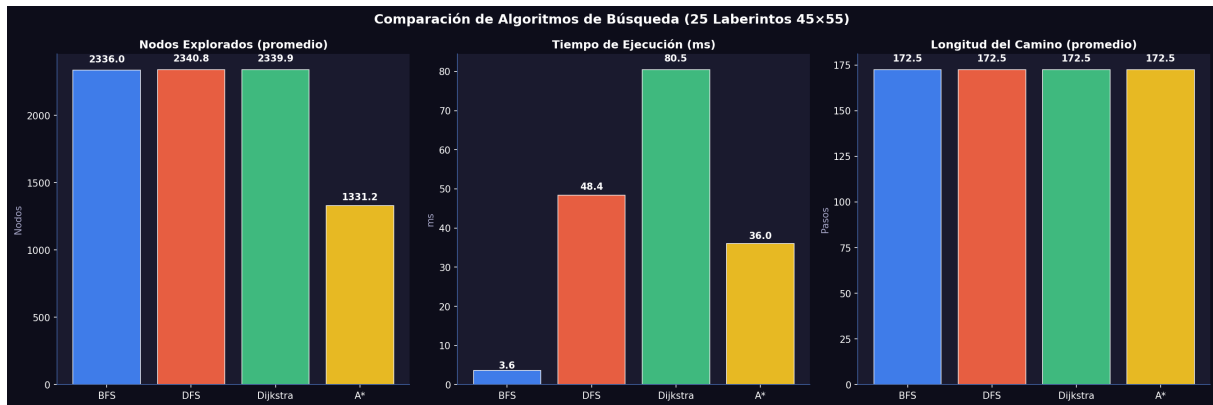


Figura 9: Comparación de métricas promedio entre algoritmos en los 25 laberintos de 45×55 celdas.

5. Conclusiones

La implementación de los algoritmos de Kruskal y Prim permitió generar laberintos perfectos de forma eficiente. Ambos producen laberintos con estructuras distintas: Kruskal distribuye las conexiones de manera más uniforme en todo el espacio, mientras que Prim genera laberintos con crecimiento más local y ramificado. Kruskal es más predecible en tiempo de ejecución gracias a la estructura Union-Find, mientras que Prim puede variar más según la celda inicial elegida.

En cuanto a los algoritmos de búsqueda, A* demostró ser el más eficiente en la mayoría de los escenarios. La heurística de distancia Manhattan le permitió reducir de forma considerable el número de nodos explorados sin sacrificar la optimalidad del camino encontrado. BFS y Dijkstra garantizan igualmente encontrar el camino más corto, pero con un costo de exploración mayor. DFS no garantiza la optimalidad y su desempeño varía bastante según la estructura del laberinto, aunque en algunos casos puede explorar menos nodos que los demás.

Los resultados del Problema 3 confirman que para laberintos con costos uniformes, A* es la mejor opción cuando se tiene una heurística admisible, seguido por BFS para casos donde no se requiere heurística. Dijkstra se comporta de forma equivalente a BFS en este tipo de grafos, y DFS es adecuado solo cuando se prioriza el uso de memoria sobre la optimalidad del camino.